

13

오디오 비디오 제어 및 위치 정보 서비스, 웹 워커

강의 목표

1. 자바스크립트와 <audio> 태그와 <video> 태그를 이용하여 오디오와 비디오 재생, 중단, 새로운 미디어 재생 등 다양한 제어를 할 수 있다.
2. HTML5에서 제공하는 위치 정보 서비스를 이해한다.
3. 자바스크립트 코드로 PC나 모바일 장치의 현재 위도 경도 위치를 알아낼 수 있다.
4. 자바스크립트 코드로 PC나 모바일 장치의 위치가 변할 때마다 변경된 위치 정보를 알아낼 수 있다.
5. HTML5에서 제공하는 백그라운드 기능의 웹 워커 개념을 이해한다.
6. 자바스크립트로 웹 워커 API를 활용하여 백그라운드 작업을 만들 수 있다.

오디오/비디오 제어

3

- HTML5의 오디오/비디오 제어
 - ▣ <audio>, <video> 태그
 - HTML5에서 플러그인의 도움 없이 오디오/비디오 삽입
 - ▣ 자바스크립트 코드로 미디어 재생/중단/ 등 미디어 제어
- <audio>와 <video> 태그

```
<audio id="audio" src="media/EmbraceableYou.mp3"
  loop controls>
  웹 브라우저가 audio 태그를 지원하지 않습니다.
</audio>
```

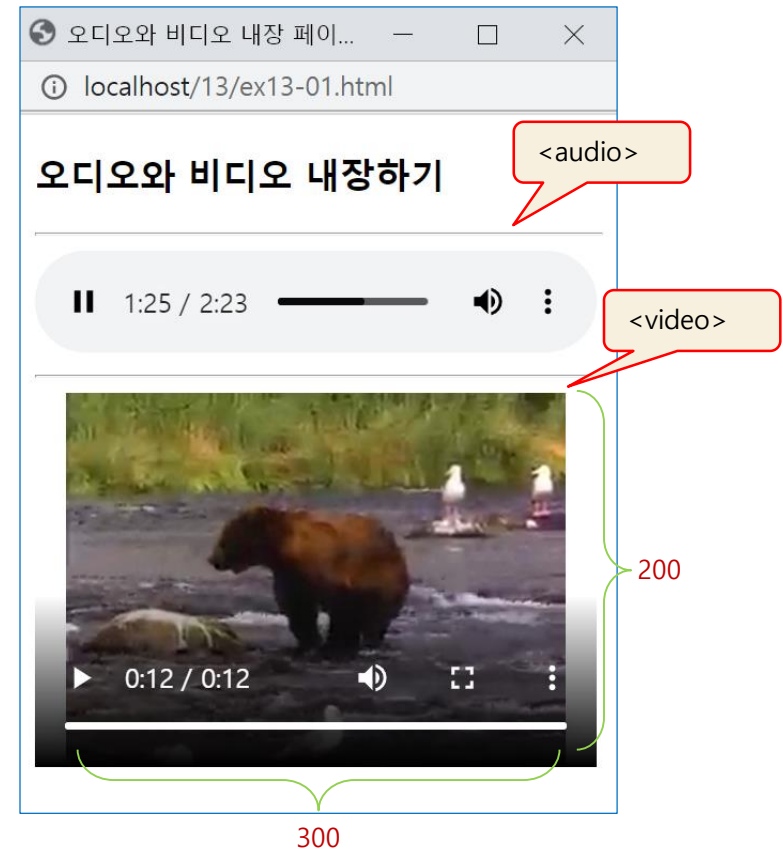
```
<video id="video" width="300" height="200"
  controls>
  <source src="media/bear.mp4" type="video/mp4">
  웹 브라우저가 video 태그를 지원하지 않습니다.
</video>
```



예제 13-1 오디오/비디오를 가진 웹 페이지

4

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>오디오와 비디오 내장 페이지</title>
</head>
<body>
<h3>오디오와 비디오 내장하기</h3>
<hr>
<audio id="audio" src="media/EmbraceableYou.mp3"
      loop controls>
  웹 브라우저가 audio 태그를 지원하지 않습니다.
</audio>
<hr>
<video id="video" width="300" height="200" controls>
  <source src="media/bear.mp4" type="video/mp4">
  웹 브라우저가 video 태그를 지원하지 않습니다.
</video>
</body>
</html>
```



자바스크립트로 오디오 제어

5

□ <audio> 태그에 로드된 오디오 제어

```
<audio id="audio" src="media/EmbraceableYou.mp3" loop controls>...</audio>
```

□ 오디오 DOM 객체 알아내기

```
let audio = document.getElementById("audio");
```

□ 오디오 재생 및 일시 중지

```
audio.play(); // 재생. 중지된 이후부터 재생  
audio.pause(); // 일시 중지
```

□ 오디오 처음부터 재생

```
audio.load(); // src에 지정된 오디오 데이터 로드  
audio.play(); // 처음부터 재생
```

□ 오디오 음량 제어와 음소거

```
audio.volume += 0.1; // 0.1 만큼 음량 증가  
audio.muted = true; // 음소거. 음량(volume) 변경 없음
```

예제 13-2 자바스크립트로 오디오제어기 만들기

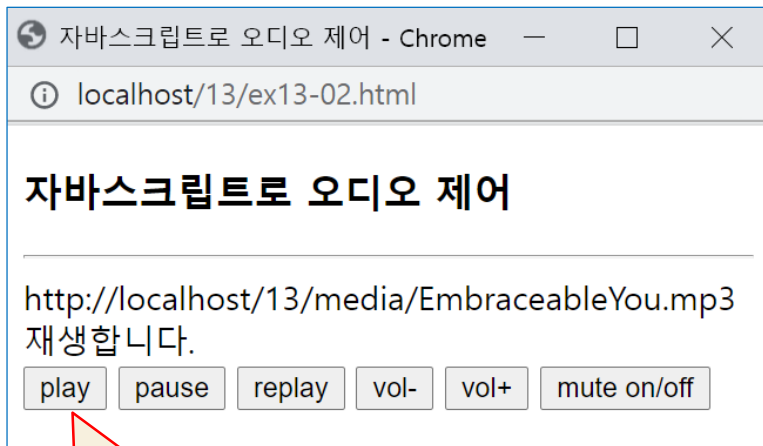
6

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>자바스크립트로 오디오 제어</title>
</head>
<body>
<h3>자바스크립트로 오디오 제어</h3>
<hr>
<audio id="audio" src="media/EmbraceableYou.mp3"></audio>
<div id="msg">이곳에 오디오 제어 메시지 출력</div>
<button type="button" id="play" onclick="control(event)">play</button>
<button type="button" id="pause" onclick="control(event)">pause</button>
<button type="button" id="replay" onclick="control(event)">replay</button>
<button type="button" id="vol-" onclick="control(event)">vol-</button>
<button type="button" id="vol+" onclick="control(event)">vol+</button>
<button type="button" id="mute on/off" onclick="control(event)">mute on/off
</button>
```

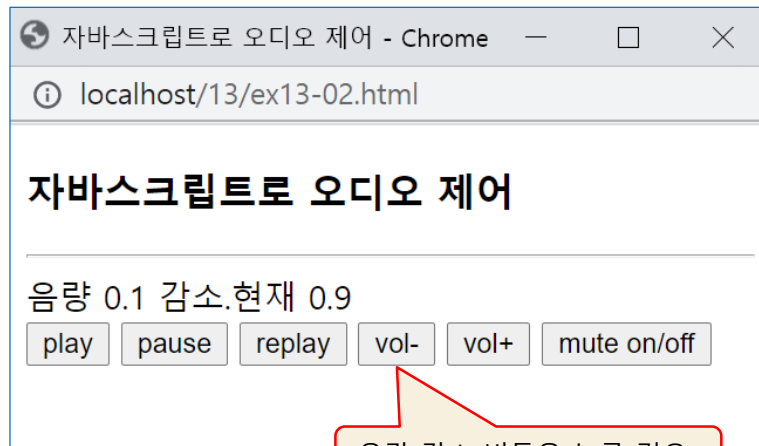
```
<script>
let div = document.getElementById("msg");
let audio = document.getElementById("audio");
function control(e) {
  let id = e.target.id;
  if(id == "play") { // play 버튼 클릭
    audio.play(); // 재생
    div.innerHTML = "재생중입니다.";
  }
  else if(id == "pause") { // pause 버튼 클릭
    audio.pause(); // 일시 중지
    div.innerHTML = "일시중지되었습니다.";
  }
  else if(id == "replay") { // replay 버튼 클릭
    audio.load(); // src에 지정된 미디어 다시 로딩
    audio.play(); // 처음부터 다시 재생
    div.innerHTML = audio.src + "를 처음부터 재생합니다.";
  }
  else if(id == "vol-") { // vol- 버튼 클릭
    audio.volume -= 0.1; // 음량 0.1 감소
    if(audio.volume < 0.1) audio.volume = 0;
    div.innerHTML = "음량 0.1 감소." + "현재 " + audio.volume;
  }
  else if(id == "vol+") { // vol+ 버튼 클릭
    audio.volume += 0.1; // 음량 0.1 증가
    if(audio.volume > 0.9) audio.volume = 1.0;
    div.innerHTML = "음량 0.1 증가." + "현재 " + audio.volume;
  }
  else if(id == "mute on/off") { // mute on/off 버튼 클릭
    audio.muted = !audio.muted; // 음소거 토글
    if(audio.muted) div.innerHTML = "음소거";
    else div.innerHTML = "음소거 해제";
  }
}
</script>
</body></html>
```

예제 13-2 실행 결과

7



재생 버튼을 누른 경우



음량 감소 버튼을 누른 경우

비디오 제어

8

□ <video> 태그에 로드된 비디오 제어

```
<video id="video" width="320" height="240" controls>...</video>
```

▣ 비디오 DOM 객체 알아내기

```
let video = document.getElementById("video");
```

▣ width, height와 videoWidth, videoHeight 프로퍼티

- width, height : <video> 태그의 width, height 속성 반영
- videoWidth, videoHeight : 비디오의 화면 해상도

▣ loadedmetadata 이벤트

- 비디오 파일의 로드 완료시, video 객체에 loadedmetadata 이벤트 발생
- 예) 비디오의 해상도 알아내기. 비디오가 로드되어야 비로소 videoWidth, videoHeight 프로퍼티가 정확한 값을 가짐

```
video.onloadedmetadata = function f(e) {  
    alert(video.videoWidth + "," + video.videoHeight);  
}
```


예제 13-3 비디오를 원본 크기로 재생

9

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>비디오 원본 크기로 출력</title>
</head>
<body>
<h3>비디오 원본 크기로 출력</h3>
<hr>
<video id="video" width="0" height="0" controls>
  <source src="media/bear.mp4" type="video/mp4">
  웹 브라우저가 video 태그를 지원하지 않습니다.
</video>
<script>
let video = document.getElementById("video");
video.onloadedmetadata = function f(e) {
  alert(video.videoWidth + "x" + video.videoHeight);
  video.width = video.videoWidth;
  video.height = video.videoHeight;
}
</script>
</body>
</html>
```

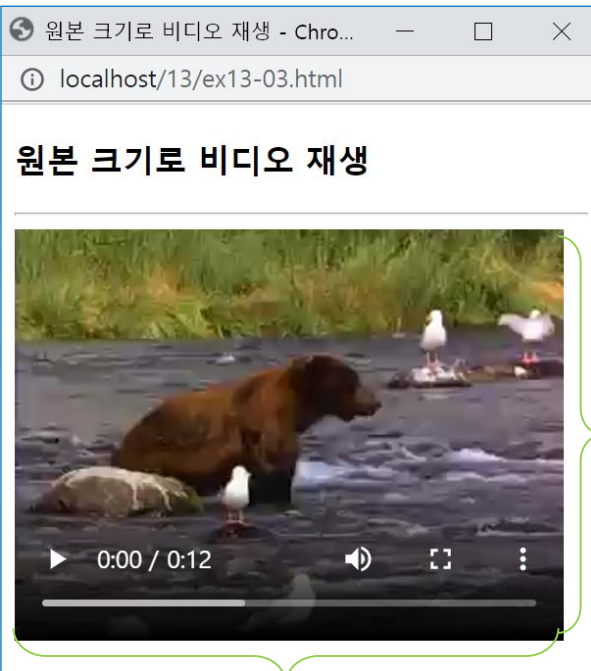
의도적인
0x0 크기

<video> 태그의 크기를
비디오의 원본 크기로 지정

localhost 내용:

320x240

확인



320

240

오디오와 비디오의 onended 리스너

10

□ onended 리스너

- ▣ 오디오/비디오의 재생이 완료되었을 때 호출되는 이벤트 리스너
- ▣ 예) 리스너 작성 사례

```
<audio id="audio" src="media/EmbraceableYou.mp3" controls> </audio>

<script>
  let audio = document.getElementById("audio");
  audio.onended = function (e) {
    // ended 이벤트 처리 코드
  }
</script>
```

- ▣ loop 속성이 설정되면 onended 이벤트 리스너 호출되지 않음

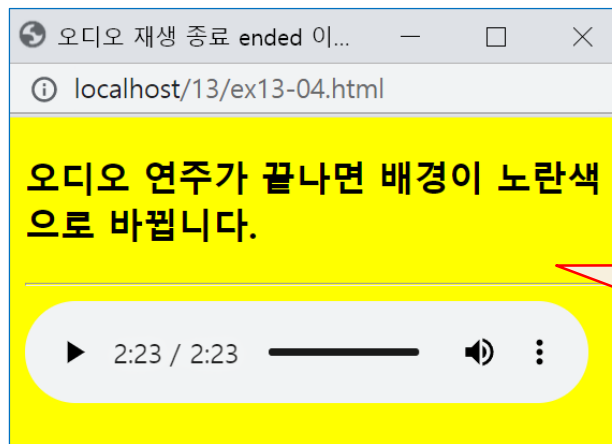
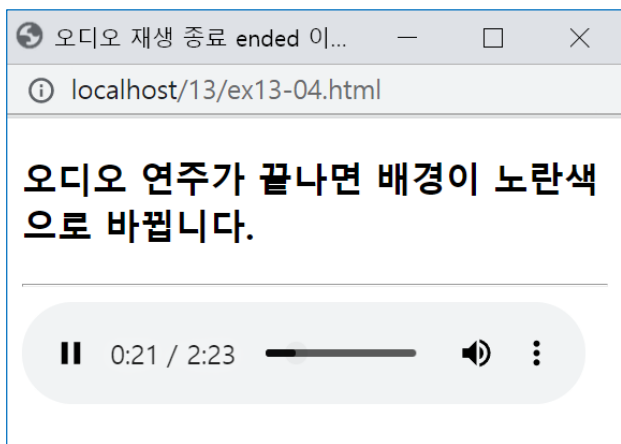
```
<audio src="..." loop> <!-- loop 속성이 있으면 ended 이벤트 발생하지 않음 -->
```

예제 13-4 오디오 재생이 끝나면 웹 페이지를 노란색으로 변경

11

onended 리스너를 활용하여 오디오 재생이 끝나면 전체 배경을 노란색으로 변경하라.

```
<!DOCTYPE html>
<html>
<head> <meta charset="utf-8">
<title>오디오 재생 종료 ended 이벤트 받기</title> </head>
<body>
<h3>오디오 연주가 끝나면 배경이 노란색으로 바뀝니다.</h3>
<hr>
<audio id="audio" src="media/EmbraceableYou.mp3" controls> </audio>
<script>
  let audio = document.getElementById("audio");
  audio.onended = function (e) { document.body.style.backgroundColor="yellow"; }
</script>
</body>
</html>
```



재생이 종료되면
ended 이벤트 발생.
배경색을 노란색으로 변경

미디어 소스 변경/미디어 로드

12

- 현재 재생 중인 미디어 변경
 - ▣ 다음 3 단계 필요
 - `audio.src="media/Aegukga.mp3";` // 새로운 미디어 지정
 - `audio.load();` // src에 지정된 미디어 새로 로딩. 생략 가능
 - `audio.play();` // 로딩된 미디어 재생

위치 정보 서비스

13

- HTML5의 위치 정보 서비스란?
 - ▣ 컴퓨터/모바일 장치(사용자)의 위치(위도와 경도)를 브라우저(자바스크립트 코드)에게 공급하는 서비스
- 보안 문제
 - ▣ 웹 브라우저를 탑재한 장치(사용자)의 위치가 웹 서버에게 알려지고
 - ▣ 다른 사용자에게 위치가 노출될 수 있는 문제점
- 위치 정보 서비스는 보안이 유지되는 경우에만 제공
 - ▣ localhost로 접속
 - 브라우저는 로컬 컴퓨터에 설치된 웹 서버에 접속하므로 다른 컴퓨터의 사용자가 웹 서버에 접속할 수 없으므로 안전
 - ▣ 웹 서버 없는 로컬 컴퓨터의 웹 페이지
 - 다른 사용자의 개입이 원천적으로 불가능하므로 안전
 - ▣ https로 접속
 - 웹 서버와 브라우저 사이의 통신은 암호화 기반 보안 통신(ssl, secure sockets layer)을 이용하므로 안전

geolocation 객체

14

□ geolocation 객체

□ 위치 정보 서비스를 제공하는 자바스크립트 객체

`navigator.geolocation`, `window.navigator.geolocation`

□ 위치 정보 서비스 2가지

- 현재 위치 서비스 : 요청 시 현재 위치를 알려주는 서비스
- 반복 위치 서비스 : 위치가 변경될 때마다 반복하여 알려주는 서비스

메소드	설명
<code>getCurrentPosition()</code>	현재 위치 얻기
<code>watchPosition()</code>	위치가 변경될 때마다 알려주는 반복 위치 서비스 시작
<code>clearWatch()</code>	반복 위치 서비스 중단

□ 브라우저의 위치 정보 서비스 지원 여부

```
if(navigator.geolocation) {  
    // 브라우저가 위치 정보 서비스를 제공한다.  
}
```

현재 위치 얻기

15

□ 현재 위치 얻기

▣ getCurrentPosition() 메소드 호출

- getCurrentPosition()은 호출 즉시 현재 위치를 리턴하는 것이 아님
- 위치가 파악되면 호출될 콜백 함수 positionCallback(Position) 등록

```
navigator.geolocation.getCurrentPosition(success); // success()를 콜백 함수로 등록
...
// 위치가 파악되면 success() 호출, 위치 정보가 있는 position 객체가 매개 변수로 전달
function success(position) {
    let lat = position.coords.latitude; // 위도
    let lon = position.coords.longitude; // 경도
    alert("현재위치(" + lat + ", " + lon + ")");
}
```


예제 13-5 getCurrentPosition()로 현재 위치 파악

16

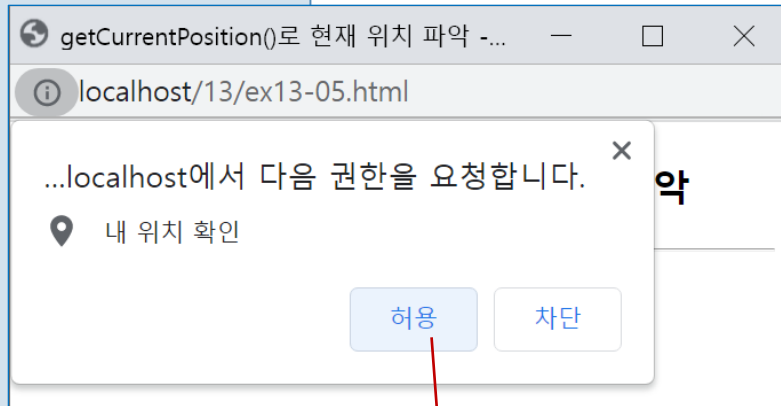
```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>getCurrentPosition()로 현재 위치 파악</title>
</head>
<body>
<h3>getCurrentPosition()로 현재 위치 파악</h3>
<hr>
<div id="msg">이곳에 위치 정보 출력</div>
<script>
if(navigator.geolocation)
    navigator.geolocation.getCurrentPosition(success); // 현재 위치 정보 요청
else
    alert("지원하지 않음");

// 위치 파악 시 success() 호출. 위치 정보가 들어 있는 position 객체가 매개 변수로 넘어온다.
function success(position) {
    let lat = position.coords.latitude; // 위도
    let lon = position.coords.longitude; // 경도
    let acc = position.coords.accuracy; // 정확도

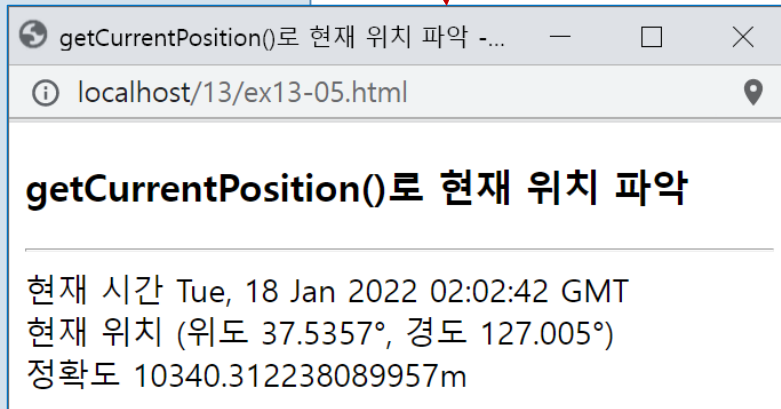
    // 위도와 경도의 소수점 이하 자리가 너무 길어 유효 숫자 6자리로 따름
    lat = lat.toPrecision(6); lon = lon.toPrecision(6);

    let now = new Date(position.timestamp); // 현재 날짜와 시간

    let text = "현재 시간 " + now.toUTCString() + "<br>";
    text += "현재 위치 (위도 " + lat + "°, 경도 " + lon + "°)<br>";
    text += "정확도 " + acc + "m<br>";
    document.getElementById("msg").innerHTML = text;
}
</script>
</body>
</html>
```



허용 버튼 클릭



웹 페이지에 지도 넣기

17

- 위치 정보 서비스와 지도 서비스는 별개
 - 위치 정보 서비스는 웹 브라우저의 구현에 달려 있음
 - 지도는 지도 서비스 기업(구글, 네이버, 카카오, 티맵 등)의 고유 자산
 - 최근, 지도 서비스 유료화

- OpenStreetMap 지도 - 개방형 무료 지도

- 1) 지도 전체 출력

<https://www.openstreetmap.org/#map=zoom레벨/위도/경도>

- 사례(예제 13-6) : 위도와 경도 (37.5823, 127.0100)인 위치를 중심, 줌 레벨 15

<https://www.openstreetmap.org/#map=15/37.5823/127.0100>

%2C는 콤마(,) 문자를
URL에 사용하기 위해
인코딩한 값

- 2) <iframe>으로 웹 페이지에 지도 내장(지도 영역을 박스로 표시)

<https://www.openstreetmap.org/export/embed.html?bbox=박스의작은경도값%2C박스의작은위도값%2C박스의큰경도값%2C박스의큰위도값>

- 사례 : (lat-0.01, lon-0.01)x(lat + 0.01, lon + 0.01) 영역의 지도 출력

```
<iframe id="map"></iframe>
```

```
...
let map = document.getElementById("map");
map.src = "https://www.openstreetmap.org/export/embed.html?bbox=" + (lon-0.01) +
"%2C" + (lat-0.01) + "%2C" + (lon+0.01) + "%2C" + (lat + 0.01);
```

예제 13-6 현재 위치의 지도를 웹 페이지에 내장(1)

18

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>현재 위치와 지도 출력</title>
</head>
<body>
<h3>현재 위치와 지도 출력</h3>
<hr>
<div id="msg">이곳에 위치 정보 출력</div>
<iframe id="map" width="425" height="350" frameborder="0" scrolling="no" marginheight="0"
marginwidth="0" ></iframe><br/>
<a id="bigmaplink" target="_blank">새 창에 큰 지도 보기</a>
<script>
if(navigator.geolocation)
    navigator.geolocation.getCurrentPosition(success); // 현재 위치 정보 요청
else
    alert("지원하지 않음");
```

다음 페이지에 소스 계속됨 ...

예제 13-6 현재 위치의 지도를 웹 페이지에 내장(2)

19

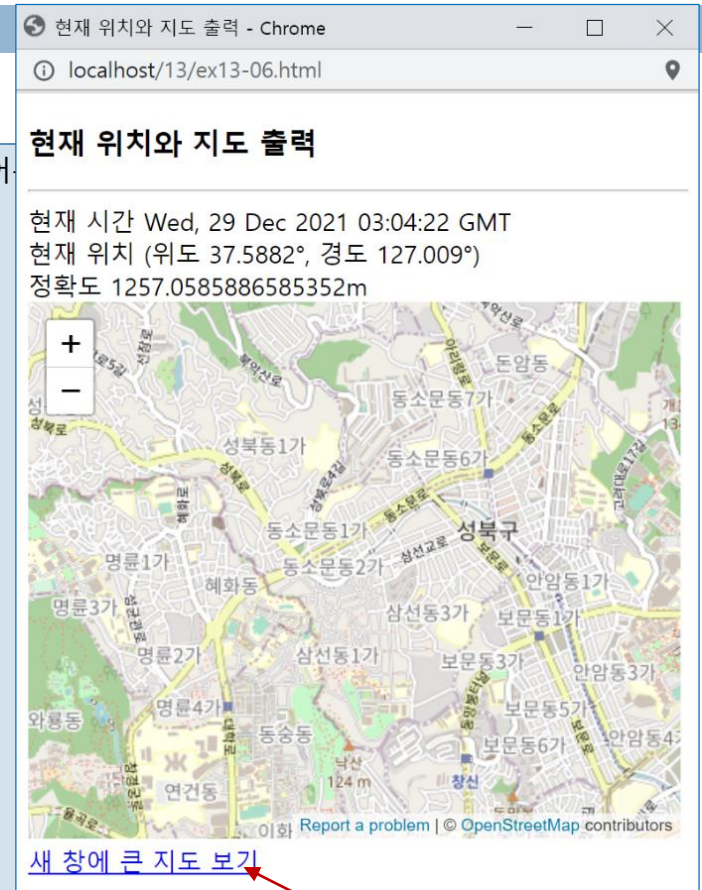
```
// 위치 파악 시 success() 호출. 위치 정보가 들어 있는 position 객체가 매개 변수로 넘어
function success(position) {
    let lat = position.coords.latitude; // 위도
    let lon = position.coords.longitude; // 경도
    let acc = position.coords.accuracy; // 정확도

    // 위도와 경도의 소수점 이하 자리가 너무 길어 유효 숫자 6자리로 자름
    lat = lat.toPrecision(6); lon = lon.toPrecision(6);

    let now = new Date(position.timestamp);
    let text = "현재 시간 " + now.toUTCString() + "<br>";
    text += "현재 위치 (위도 " + lat + "°, 경도 " + lon + "°)<br>";
    text += "정확도 " + acc + "m<br>";
    document.getElementById("msg").innerHTML = text;

    let map = document.getElementById("map");
    map.src = "https://www.openstreetmap.org/export/embed.html?bbox=" +
        (parseFloat(lon)-0.01) + "%2C" + (parseFloat(lat)-0.01) + "%2C" +
        (parseFloat(lon)+0.01) + "%2C" + (parseFloat(lat) + 0.01);
    // lat와 lon은 문자열이므로 숫자로 바꾸기 위해 parseFloat() 사용

    let maplink = document.getElementById("bigmaplink");
    let zoom = 15; // 지도의 줌 레벨. 숫자가 클수록 자세한 지도
    maplink.href = "https://www.openstreetmap.org/#map=" + zoom + "/" + lat + "/" + lon;
}
</script>
</body>
</html>
```

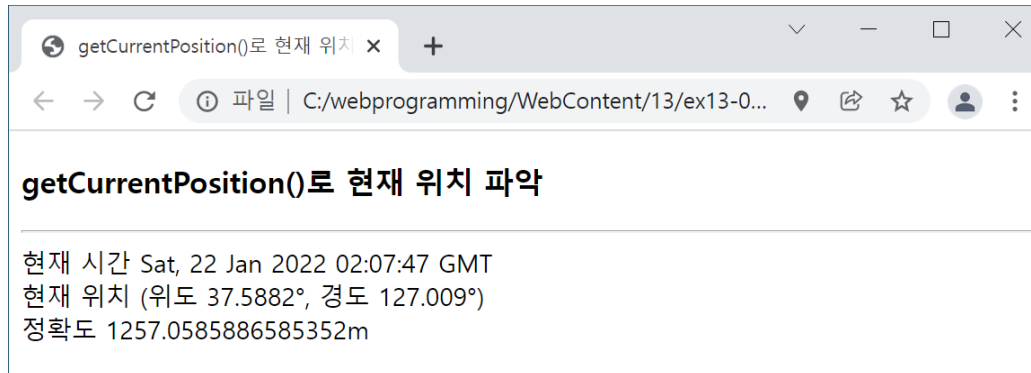
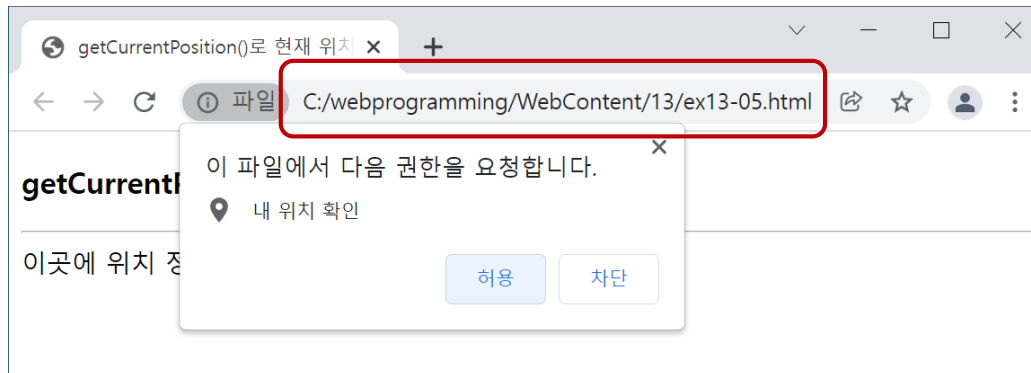


클릭하면 다음 슬라이드처럼
큰 지도 출력

참고 – 예제 13-5를 로컬 컴퓨터에서 실행하기

21

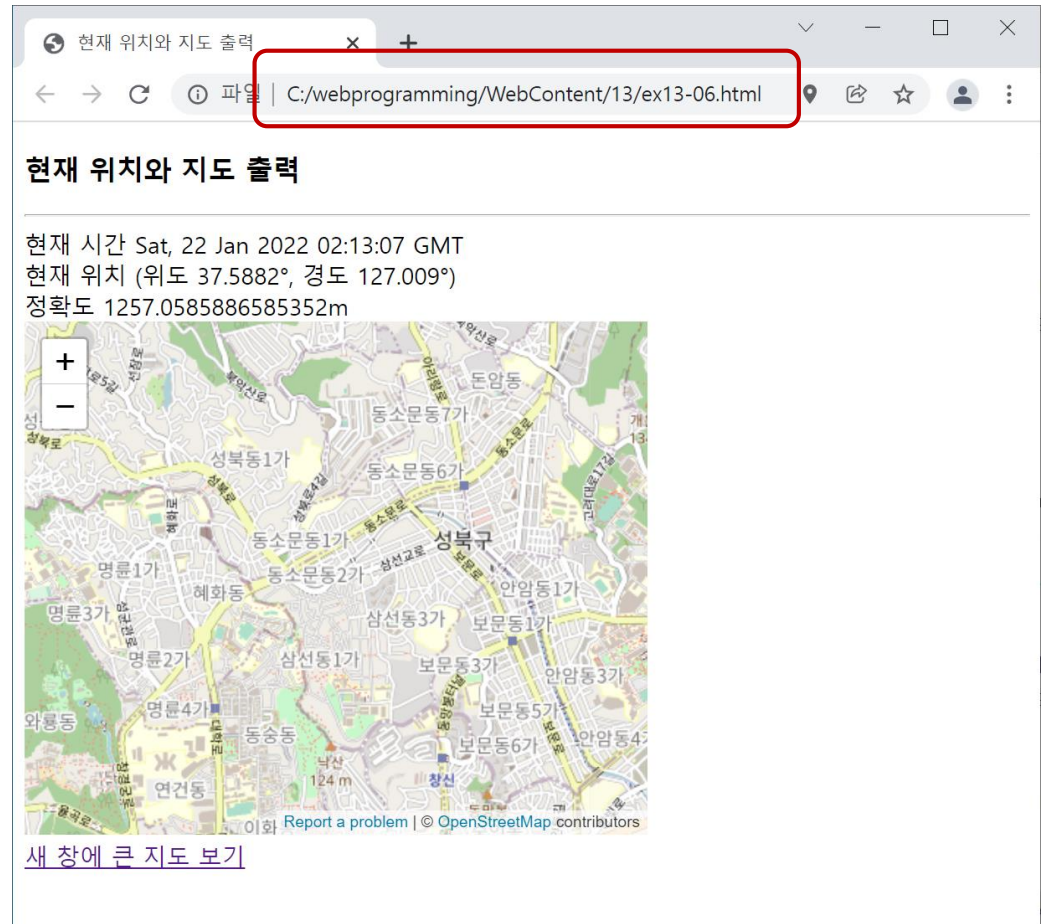
- 예제 13-5를 로컬 컴퓨터의 적절한 위치에 저장
 - ▣ 저자의 경우 C:/webprogramming/WebContent/13/ex13-05.html로 저장
- 크롬에서 직접 실행
 - ▣ 탐색기에 ex13-05.html 파일을 더블클릭



참고 – 예제 13-6을 로컬 컴퓨터에서 실행하기

22

- 예제 13-6를 로컬 컴퓨터의 적절한 위치에 저장
 - ▣ 저자의 경우 C:/webprogramming/WebContent/13/ex13-06.html로 저장
- 크롬에서 직접 실행
 - ▣ 탐색기에 ex13-06.html 파일을 더블클릭



반복 위치 서비스

23

□ 위치 변경 시마다 현재 위치 얻기

▣ watchPosition() 호출

- 위치가 변경될 때마다 호출되는 콜백 함수 등록
- watchPosition()의 리턴 값 : 반복 위치 서비스 id

```
let watchID = navigator.geolocation.watchPosition(changed); // changed()를
// 콜백 함수로 등록하고, 반복 위치 서비스 시작

...
// 위치가 바뀌면 changed() 호출, 위치 정보가 있는 position 객체가 매개 변수로 전달
function changed(position) {
  let lat = position.coords.latitude; // 변경된 위도
  let lon = position.coords.longitude; // 변경된 경도
  alert("(" + lat + ", " + lon + ") 위치로 변경됨");
}
```

□ 반복 위치 서비스 중단

▣ clearWatch() 호출

- watchPosition()가 리턴한 id로 반복 위치 서비스 중단

```
navigator.geolocation.clearWatch(watchID); // watchID의 반복 위치 서비스 중단
```


예제 13-7 반복 위치 서비스

```
<!DOCTYPE html>
<html>
<head><meta charset="utf-8"><title>watchPosition()으로 반복 위치 서비스</title></head>
<body>
<h3>watchPosition()으로 반복 위치 서비스</h3><hr>
<div id="msg">이곳에 위치 정보 출력</div>
<iframe id="map" width="425" height="350" frameborder="0" scrolling="no" marginheight="0" marginwidth="0" ></iframe><br/>
<script>
let options = { // watchPosition()의 마지막 매개 변수로 전달할 객체
    enableHighAccuracy: false,
    timeout: 5000,
    maximumAge: 0 };
let count=0; // 반복 위치 서비스가 호출되는 횟수
let watchID;

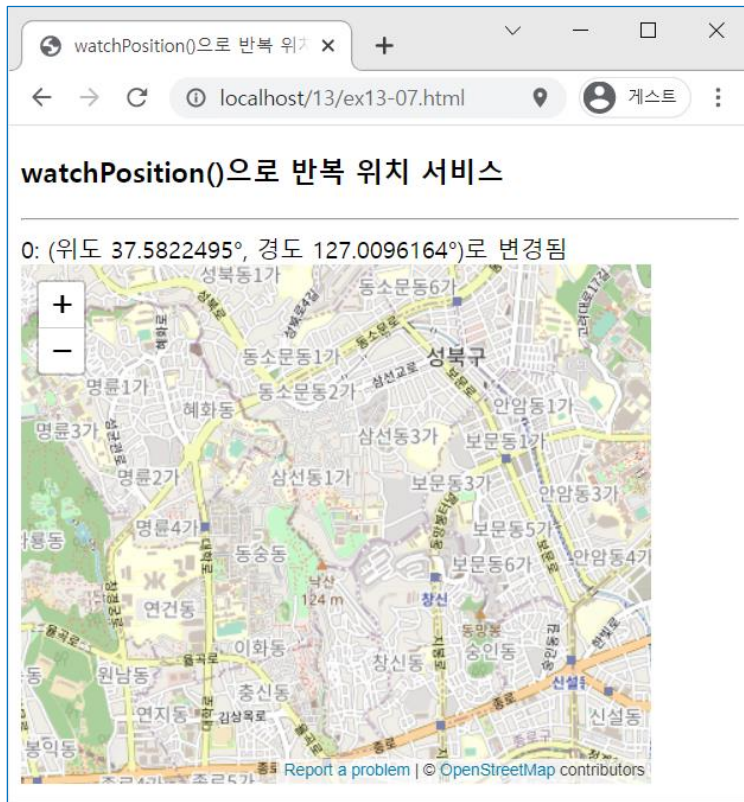
if(navigator.geolocation) {
    // changed() 콜백 함수를 등록하고, 반복 위치 서비스 시작
    watchID = navigator.geolocation.watchPosition(changed, null, options);
}
else { alert("지원하지 않음"); }

//위치가 바뀌면 changed()가 호출되고, 위치 정보가 들어 있는 position 객체가 매개 변수로 넘어온다.
function changed(position) {
    if(count == 5) { // clearWatch() 테스트를 위해 5번만 서비스
        navigator.geolocation.clearWatch(watchID); // 반복 서비스 종료
        document.getElementById("msg").innerHTML = "위치 서비스 종료";
        return;
    }
    let lat = position.coords.latitude; // 변경된 위도
    let lon = position.coords.longitude // 변경된 경도
    let text = count + ": (위도 " + lat + "°, 경도 " + lon + "°)로 변경됨<br>";
    document.getElementById("msg").innerHTML = text; // 위치 정보 출력

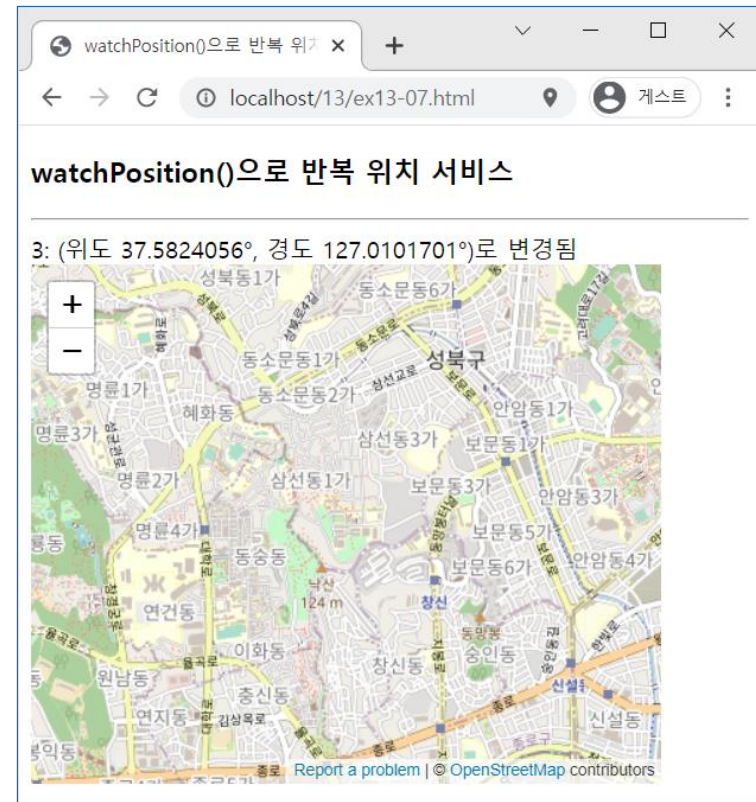
    let map = document.getElementById("map");
    map.src = "https://www.openstreetmap.org/export/embed.html?bbox=" +
        (parseFloat(lon)-0.01) + "%2C" + (parseFloat(lat)-0.01) + "%2C" +
        (parseFloat(lon)+0.01) + "%2C" + (parseFloat(lat) + 0.01); // lat와 lon은 문자열이므로 숫자로 바꾸기 위해 parseFloat() 사용
    count++; // 갱신 회수 증가
}
</script></body></html>
```

예제 13-7 실행 결과

25



처음 위치



이동 중에 변경된 위치

* 노트북을 들고 이동하면 위치가 변경되는 것을 볼 수 있다.

- 웹 워커(Web Workers)란?
 - ▣ 백그라운드 태스크를 만드는 기능
 - 자바스크립트 코드를 백그라운드에서 별도로 실행시킬 수 있는 HTML5 표준 기능
 - 백그라운드 태스크를 워커 태스크라고 부름
 - ▣ 실행 시간이 긴 계산 작업에 적합
 - 워커 태스크는 윈도우와 사용자 인터페이스 사용 불가능
- 웹 워커의 특징
 - ▣ 백그라운드 태스크로 실행할 자바스크립트 코드는 파일 형태로 저장
 - ▣ 동일 도메인(same origin) 원칙 준수
 - 자바스크립트 파일은 웹 페이지와 동일한 웹 사이트에 저장
 - ▣ 로컬 컴퓨터의 웹 페이지에서는 작동하지 않음
 - 웹 서버 설치 및 작동 필요(부록 참고)

워커 객체와 워커 태스크

27

- 워커 태스크(Worker Task)
 - ▣ 웹 워커 기능을 이용하여 만든 백그라운드 태스크
- 워커 태스크 만들기
 1. 워커 태스크를 만들 자바스크립트 코드(add1to10.js) 준비

```
// 1에서 10까지 더하고 결과 전송
let sum = 0;
for(let i=1; i<=10; i++) {
    sum += i;
}
postMessage(sum); // sum을 메인 태스크에 전송
```

2. 워커 태스크와 워커 객체 생성

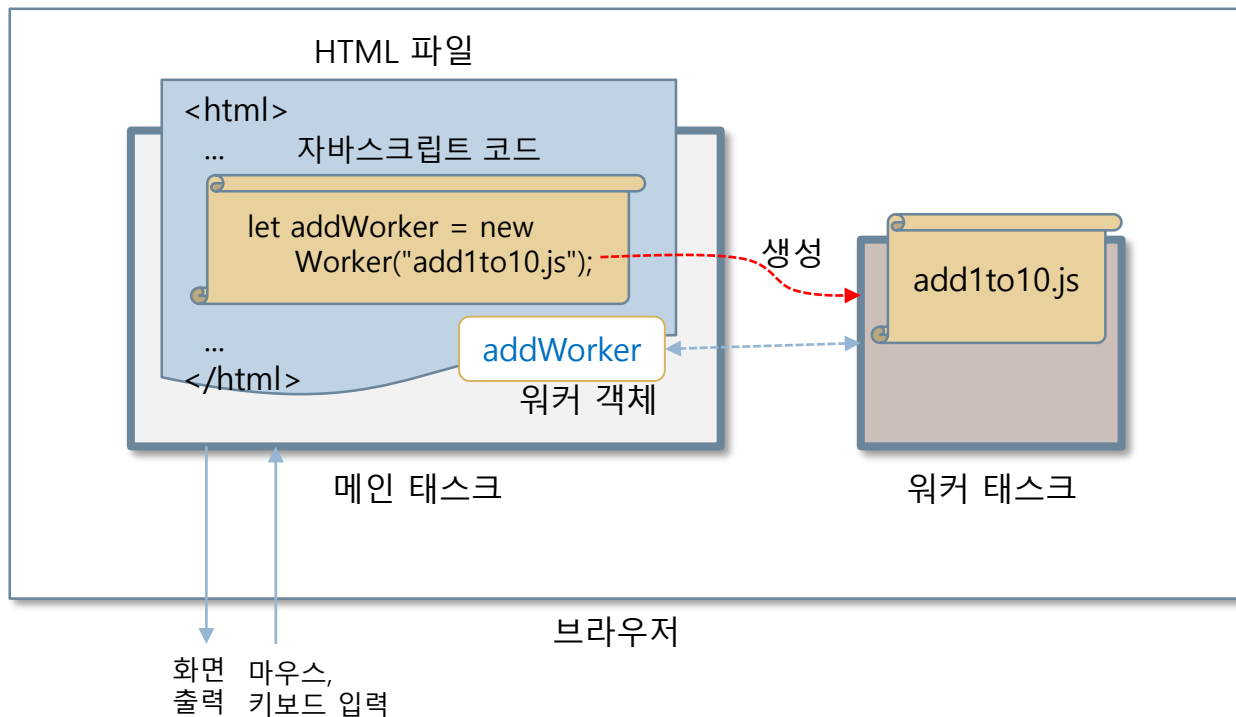
```
let addWorker = new Worker("add1to10.js");
```

- addWorker - 워커 객체
- new Worker("add1to10.js") - 워커 태스크 생성

let addWorker = new Worker("add1to10.js");

28

- 워커 태스크
 - ▣ 독립적으로 실행되는 작업 단위
- 워커 객체
 - ▣ 워커 태스크로부터 이벤트와 데이터 주고 받기
 - ▣ 워크 태스크 중단 등, 워커 태스크를 제어하는 객체



브라우저의
메인 태스크와
워커 태스크로 구성된
멀티 태스킹

워커(Worker) 객체의 메소드와 이벤트 리스너

29

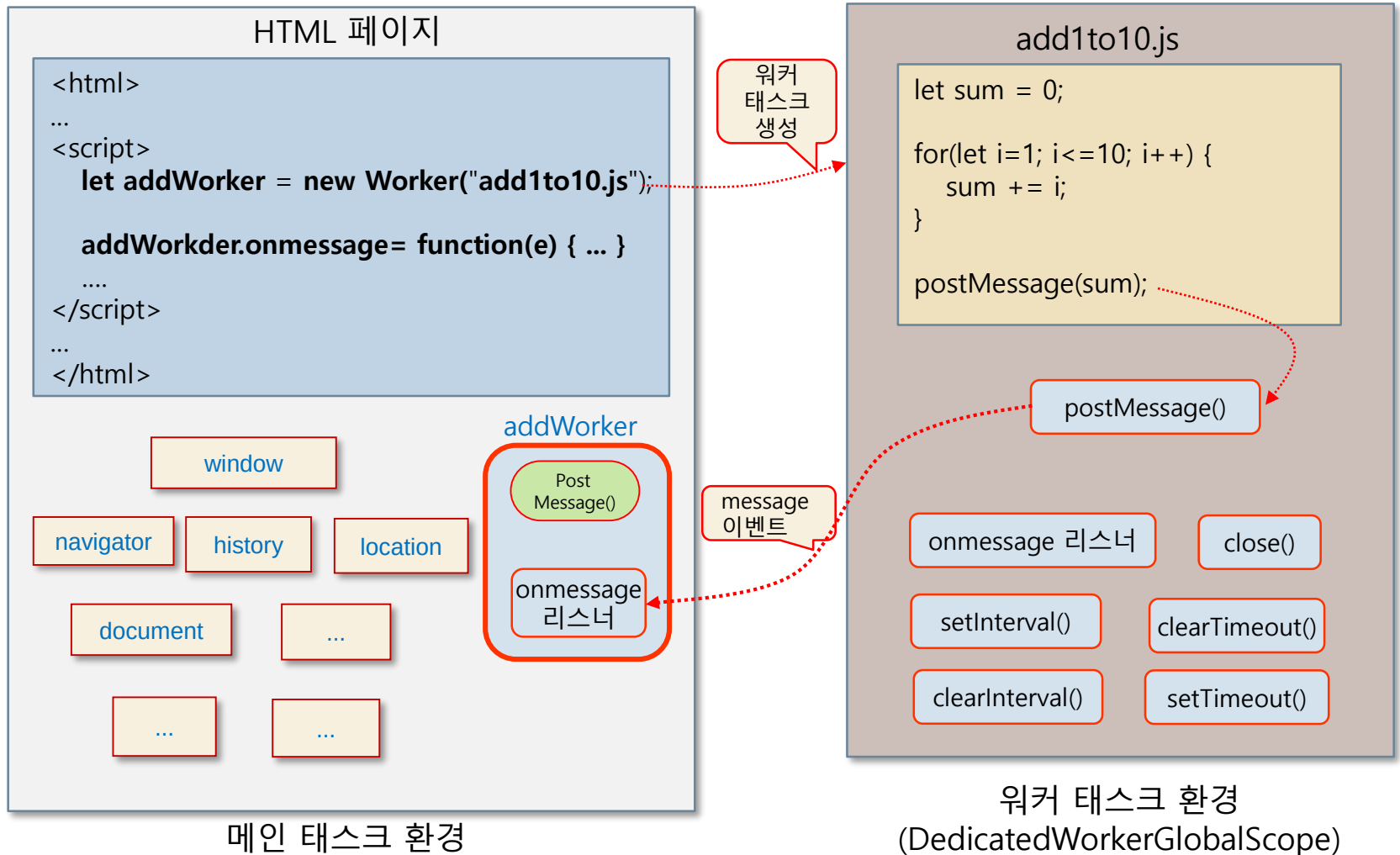
메소드	설명
Worker()	워커 태스크 생성
postMessage()	워커 태스크에 메시지 전송. 워커 태스크에는 message 이벤트 발생
terminate()	즉각 워커 태스크의 실행을 종료시킴

이벤트 리스너	설명
onmessage	워커 태스크로부터 발생한 message 이벤트를 받는 리스너
onerror	오류가 발생할 때 받는 이벤트 리스너

워커 태스크의 실행 환경

30

- 워커 태스크는 UI를 사용할 수 없는 별도의 실행 환경



워커 태스크에서 워커 객체로 message 이벤트 보내기

31

HTML 페이지

```
<html>
...
<script>
  let addWorker = new Worker("add1to10.js");

  addWorker.onmessage= function e {
    alert(e.data);
  }
</script>
...
</html>
```

addWorker 객체

```
.....
onmessage = function (e) {
  alert(e.data);
}
```

e data "55"
이벤트 객체

브라우저 윈도우

브라우저의 메인 태스크

워커
태스크
생성

add1to10.js

```
let sum = 0;

for(let i=1; i<=10; i++) {
  sum += i;
}

postMessage(sum);
```

postMessage()

워커 태스크를 생성한
메인 태스크로 메시지 보냄

DedicatedWorkerGlobalScope

워커 태스크(백그라운드 태스크) 환경

localhost 내용:

55

확인

message 이벤트 보내는 과정

32

□ 워커 태스크에서 postMessage() 호출

▣ 매개 변수에 보내고자 하는 데이터 전달

```
postMessage(sum);    // sum = 55
```

▣ 데이터를 MessageEvent 객체로 만들어 전달

- 워커 객체에서 MessageEvent 객체를 통해 데이터 수신
- MessageEvent 객체의 프로퍼티

프로퍼티	설명	r/w
data	전달되는 값으로서 문자열이거나 객체	r

□ 워커 객체의 onmessage 리스너

▣ onmessage 리스너

- 워커 태스크가 보내는 message 이벤트를 받는 코드

```
addWorker.onmessage = function (e) { // e에 MessageEvent 객체가 전달  
    alert(e.data); // e.data는 "55"  
}
```

예제 13-8 1~10까지 더하는 워커태스크 만들기

33

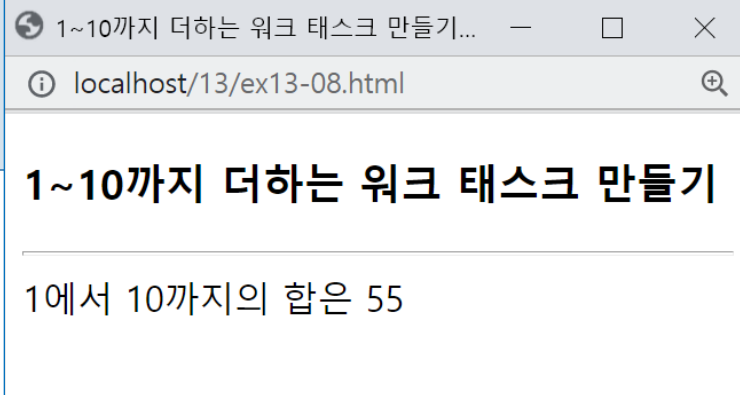
```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>1~10까지 더하는 워크 태스크 만들기</title>
</head>
<body>
<h3>1~10까지 더하는 워크 태스크 만들기</h3>
<hr>
<div>1에서 10까지의 합은 <span id="sum"></span></div>
<script>
  // addWorker 워커 객체 생성 및 워커 태스크 시작
  let addWorker = new Worker("add1to10.js");

  // 워크 태스크로부터 message 이벤트 수신
  addWorker.onmessage = function (e) { // e는 MessageEvent 객체
    // 이벤트 객체의 data(합) 출력
    document.getElementById("sum").innerHTML = e.data;
  }
</script>
</body>
</html>
```

add1to10.js

```
// 1~10까지 합 계산
let sum=0;
for(let i=1; i<=10; i++) {
  sum += i;
}

// 합을 메시지로 전송
postMessage(sum);
```



```
<script>
```

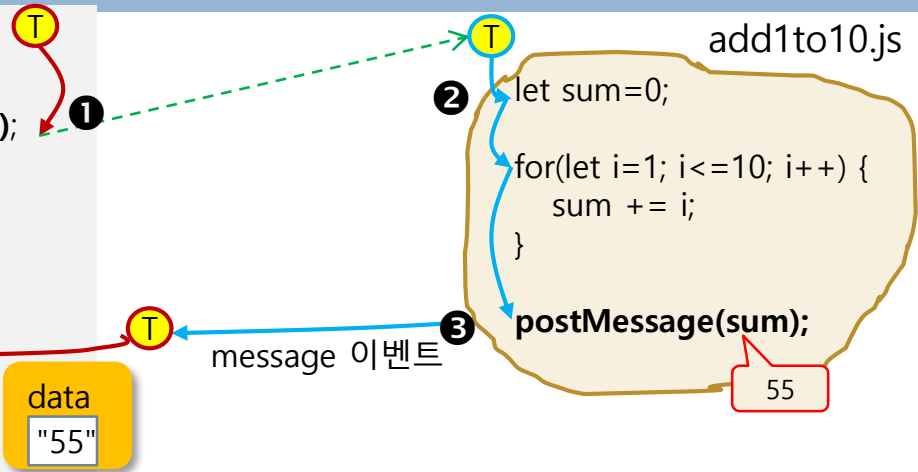
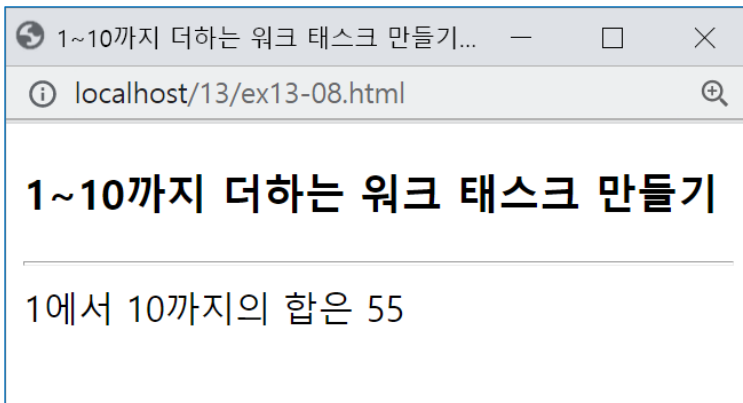
```
let addWorker = new Worker("add1to10.js");
```

```
... 다른 작업 진행 ...
```

```
addWorker.onmessage = function (e) {
  document.getElementById("sum")
    .innerHTML = e.data;
}
```

```
</script>
```

실행 결과

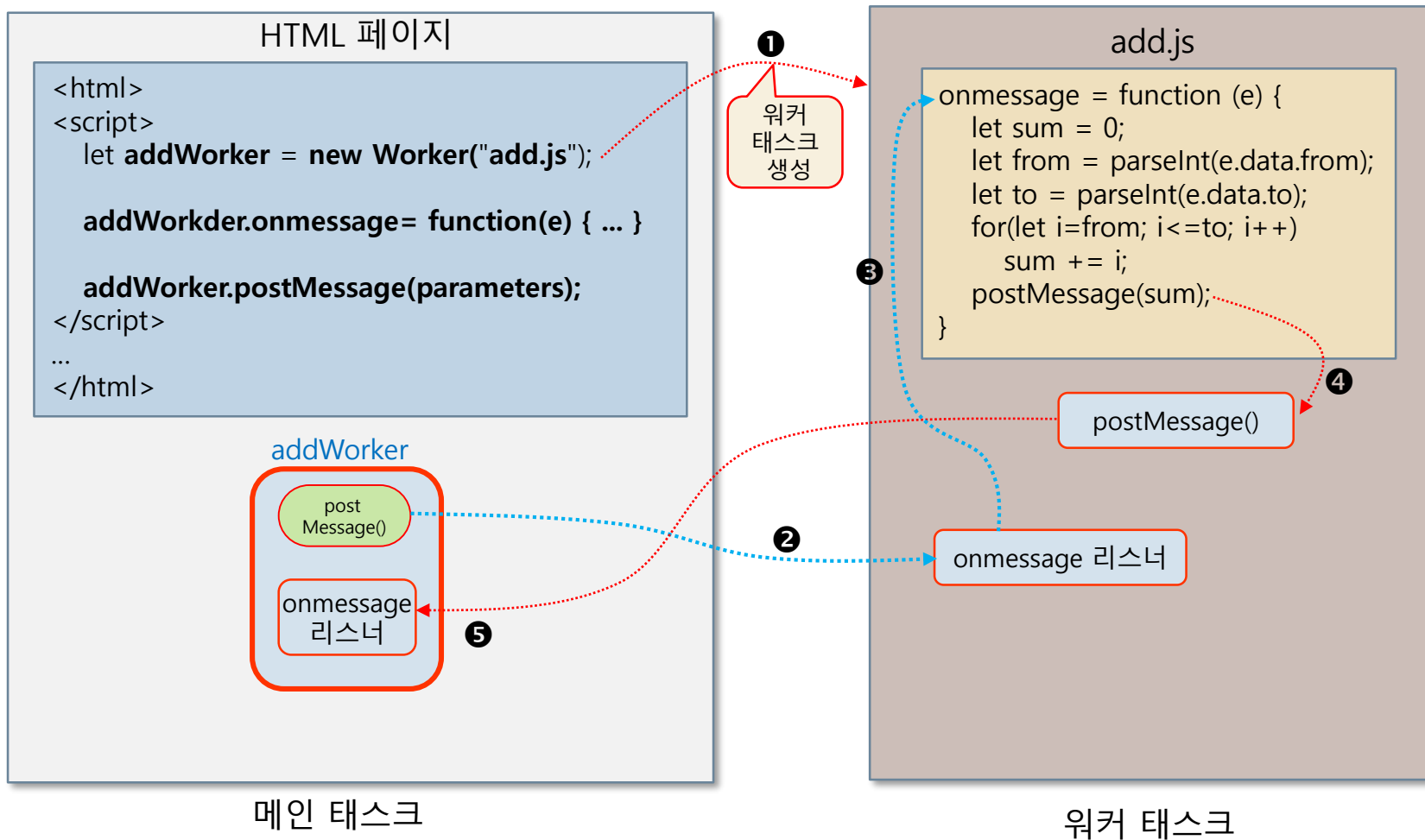


Ⓣ : 브라우저 메인 태스크
Ⓣ : 워커 태스크

메인 태스크에서 워커 태스크로 message 이벤트 보내기

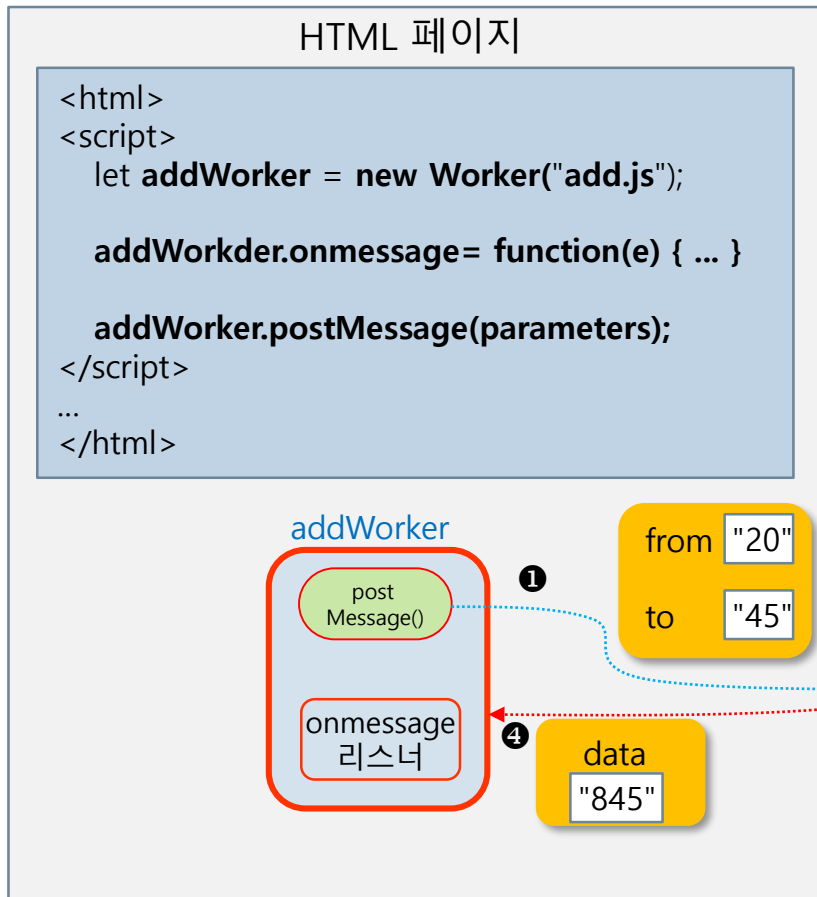
35

let addWorker = new Worker("add.js");의 실행으로형성된 메인 태스크와 워커 태스크

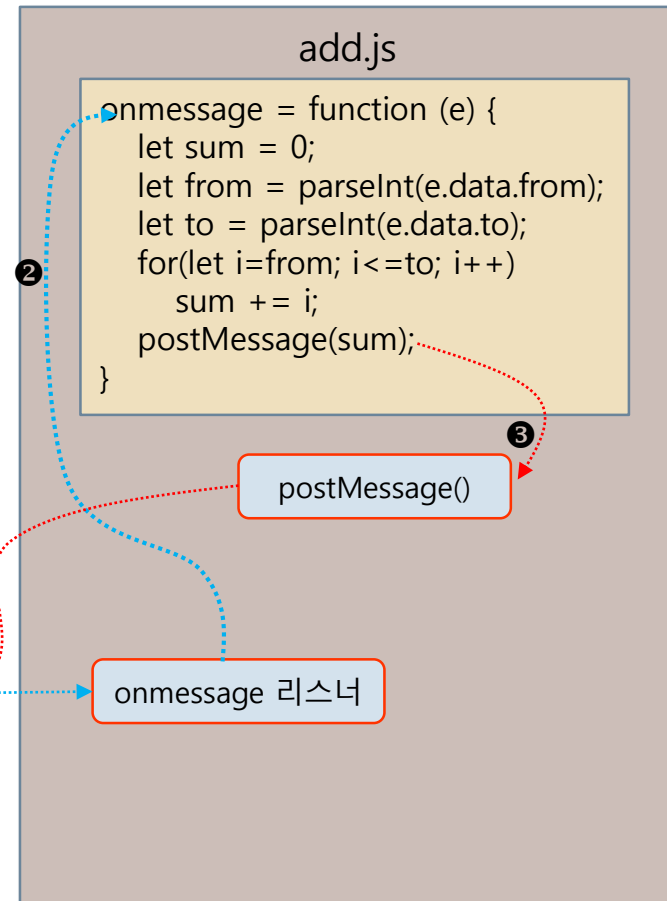


메인 태스크와 워커 태스크 사이의 데이터 전송

36



메인 태스크



워커 태스크

예제 13-9 워커 태스크에 시작 숫자와 끝 숫자를 보내고 합을 전달받는 코드

37

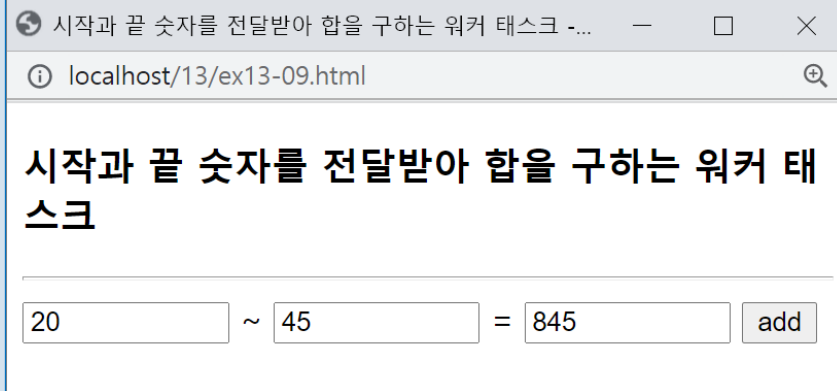
```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>시작과 끝 숫자를 전달받아 합을 구하는 워커 태스크</title>
</head>
<body>
<h3>시작과 끝 숫자를 전달받아 합을 구하는 워커 태스크</h3>
<hr>
<input id="from" type="text" size="10"> ~
<input id="to" type="text" size="10"> =
<input id="sum" type="text" size="10">
<button type="button" id="add" onclick="send()">add</button>
<script>
  let addWorker = new Worker("add.js"); // 워커 태스크 생성

  function send() { // 워크 태스크에 시작 숫자와 끝 숫자 전송
    let parameters = { // 시작 숫자와 끝 숫자로 구성된 객체
      from: document.getElementById("from").value,
      to: document.getElementById("to").value
    };
    // 시작 숫자와 끝 숫자를 담은 객체를 워커 태스크로 전송
    addWorker.postMessage(parameters);
  }

  // 워커 태스크로부터 결과를 기다리는 리스너 등록
  addWorker.onmessage = function (e) {
    // 워커 태스크로부터 전달받은 합 출력
    document.getElementById("sum").value = e.data;
  }
</script>
</body></html>
```

add.js

```
onmessage = function (e) {
  let sum = 0;
  let from = parseInt(e.data.from);
  let to = parseInt(e.data.to);
  for(let i=from; i<=to; i++)
    sum += i;
  postMessage(sum);
}
```



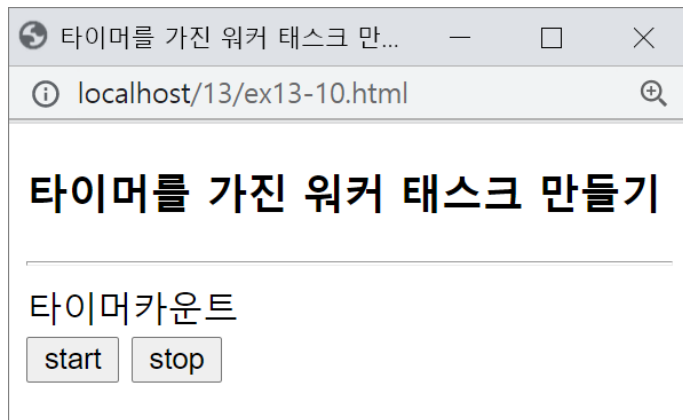
워커 태스크 종료

38

- 워크 태스크 스스로 종료
 - ▣ `close()`
- 워커 객체가 워커 태스크를 강제 종료
 - ▣ `terminate()` 메소드
 - ▣ 예) `addWorker.terminate();`
- 워커 태스크가 종료하면 워커 객체는 더 이상 워커 태스크와 `message` 이벤트를 주고받을 수 없다.

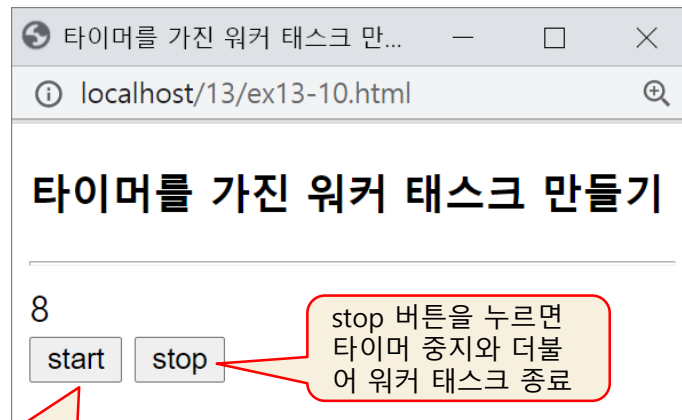
예제 13-10 1초 단위로 메시지를 보내는 워커 태스크 만들기

39



초기 화면

start 버튼을 누르면
한 번만 타이머 작동



start 버튼을 누르면 타이머 작동

예제 13-10의 코드

40

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>타이머를 가진 웹 워커 만들기</title>
</head>
<body>
<h3>타이머를 가진 웹 워커 만들기</h3>
<hr>
<div><span id="timer">타이머카운트</span></div>
<button type="button" id="start" onclick="start()">start</button>
<button type="button" id="stop" onclick="stop()">stop</button>
<script>
let addWorker = new Worker("timer.js"); // 워커 태스크 생성

addWorker.onmessage = function (e) {
  document.getElementById("timer").innerHTML = e.data;
}

function start() {
  addWorker.postMessage("start");
}

function stop() {
  addWorker.postMessage("stop");
}
</script>
</body>
</html>
```

timer.js

```
let count=1;
let timerID=null; // 타이머 ID

onmessage = function (e) { // 브라우저로부터 메시지 수신
  if(e.data == "start") {
    if(timerID != null) return; // 타이머 작동중이면 리턴
    timerID = setInterval(myCallback, 1000); // 1초 간격 myCallback() 호출
  }
  else if(e.data == "stop") {
    if(timerID == null) return; // 타이머 작동하지 않으면 리턴
    clearInterval(timerID);
    close(); // 워커 태스크 종료. 더 이상 메시지 받지 않음
  }
}

function myCallback() { // 1초 간격으로 호출
  postMessage(count); // 카운트 값을 브라우저로 전송
  count++; // 카운트 값 증가
}
```